

Flight Delay Big Data Analysis - Final Report

Luca Foresti

2026-05-21

Contents

1	Executive Summary	2
2	Assignment Coverage	2
2.1	Requirement-To-Output Mapping	3
3	Dataset And Preparation	3
3.1	Canonical Fields	4
3.2	Cleaning Policy	4
3.3	Generated Input Sizes	4
4	Analyses And Implementations	5
4.1	Analysis 1: Delay By Airport, Month, And Delay Range	5
4.2	Analysis 2: Airline-Airport Ranking	5
4.3	Spark SQL	6
4.4	Spark Core	6
4.5	Hive	6
4.6	MapReduce Stretch	7
5	Docker Standalone Simulation	7
6	Environment And Runtime Configuration	7
7	Benchmark Evidence	8
7.1	Benchmark Pivot	8
7.2	Rows Per Second	9
7.3	Speedup Ratios	9
7.4	Normalized Scalability	10
7.5	Benchmark Charts	11
8	AWS EMR Cluster Experiment	13
8.1	Cluster Size Variation	13
9	Critical Discussion	14
10	Reproducibility And Validation	14
11	Limitations	15
12	Conclusion	15
13	Evidence Appendix: First 10 Result Rows	15
13.1	Spark SQL Samples	15

13.1.1	Delay Metrics	15
13.1.2	Delay Top-Three Causes	16
13.1.3	Ranking Metrics	16
13.1.4	Ranking Comparison	16
13.2	Spark Core Samples	17
13.2.1	Delay Metrics	17
13.2.2	Delay Top-Three Causes	17
13.2.3	Ranking Metrics	17
13.2.4	Ranking Comparison	17
13.3	Hive Samples	18
13.3.1	Delay Metrics	18
13.3.2	Delay Top-Three Causes	18
13.3.3	Ranking Metrics	18
13.3.4	Ranking Comparison	19
13.4	MapReduce Stretch Samples	19

1 Executive Summary

This project analyzes the 2024 United States flight delay dataset with a reproducible big-data workflow. The repository prepares a canonical cleaned Parquet dataset, implements two analytical jobs with three required big-data technologies plus an optional MapReduce stretch, validates the outputs, records benchmark evidence, and produces the tables and figures included in this PDF.

Repository: <https://github.com/Forest904/flight-delay-big-data-analysis.git>

The implemented analyses are:

- **Analysis 1:** delay report by departure airport, month, and departure-delay range.
- **Analysis 2:** airline-airport ranking by average departure delay.

The three selected required technologies are Spark SQL, Spark Core RDDs, and Hive. Spark SQL is used as the correctness reference because its declarative aggregations and window functions make the required output rules easiest to review. Spark Core reimplements the same logic with lower-level RDD transformations. Hive provides the required SQL-on-Hadoop comparison point. M6 adds Hadoop Streaming MapReduce as a validated stretch extension, kept separate from the grade-critical default workflow.

The main reproducibility commands are:

```
make setup
make check-env
make prepare
make generate-sizes
make run-spark-sql
make run-spark-core
make run-hive
make run-mapreduce
make benchmark-local
make benchmark-mapreduce-local
make benchmark-docker-simulation
make charts
make report
```

The `make benchmark-docker-simulation` target runs single-host Docker standalone simulation evidence.

2 Assignment Coverage

Final-report requirement	Where it is answered
Data preparation operations	Dataset And Preparation
Implementation choices and pseudocode	Analyses And Implementations
First 10 result rows	Evidence Appendix
Execution-time tables and charts	Benchmark Evidence
Local execution and Docker standalone simulation settings	Docker Standalone Simulation and Benchmark Evidence
Critical discussion	Critical Discussion
GitHub repository link	Executive Summary

2.1 Requirement-To-Output Mapping

The two selected analyses use stable output schemas so every assignment requirement can be traced to concrete columns in the generated CSV files and first-10 report samples.

Analysis 1 requirement	Output column(s)
Departure airport	<code>origin_airport</code>
Month	<code>month</code>
Departure-delay range	<code>delay_range</code>
Number of flights	<code>flight_count</code>
Average departure delay	<code>avg_departure_delay</code>
Average arrival delay	<code>avg_arrival_delay</code>
Three most frequent delay or cancellation causes	<code>top_1_cause</code> , <code>top_1_count</code> , <code>top_2_cause</code> , <code>top_2_count</code> , <code>top_3_cause</code> ,

Analysis 2 requirement	Output column(s)
Departure airport	<code>origin_airport</code>
Airline	<code>airline</code>
Number of flights	<code>flight_count</code>
Average departure delay	<code>avg_departure_delay</code>
Average arrival delay	<code>avg_arrival_delay</code>
Cancellation rate	<code>cancellation_rate</code>
Airport average departure delay	<code>airport_avg_departure_delay</code>
Difference from airport average	<code>difference_from_airport_avg_departure_delay</code>
Airline rank at airport	<code>rank_at_airport</code>

The `airline` output field is the normalized `airline_code`. The raw dataset does not provide a stable airline-name field, so code-based airline identity is used consistently across Spark SQL, Spark Core, Hive, and the MapReduce stretch.

3 Dataset And Preparation

The source dataset is Kaggle `hrishitpatil/flight-data-2024`, stored locally as `data/raw/flight_data_2024.csv`. The raw CSV has 7,079,081 rows, 35 columns, and a local file size of 1,309,010,752 bytes. The raw file is not committed to Git because it is large and externally hosted.

The preparation pipeline is implemented by `src/preparation/prepare_spark.py` and is run with:

```
make prepare
```

The prepared output is `data/prepared/flights_2024_clean.parquet`. This Parquet dataset is the canonical input for Spark SQL, Spark Core, and Hive. MapReduce uses a canonical CSV export derived from the

same prepared Parquet input, not the raw Kaggle CSV. Using one prepared source keeps the comparison fair: each implementation receives the same normalized schema, the same row-retention policy, and the same benchmark input files.

3.1 Canonical Fields

The prepared dataset keeps only the fields needed by the selected analyses:

- Date and grouping: `flight_date`, `month`.
- Airline fields: `airline_code`, plus nullable `airline_name` because the raw dataset has no stable airline-name field.
- Airport fields: `origin_airport`, `destination_airport`.
- Delay metrics: `departure_delay`, `arrival_delay`.
- Cancellation and diversion: `cancelled`, `diverted`, `cancellation_code`.
- Delay causes: carrier, weather, NAS, security, and late-aircraft delay minutes.

3.2 Cleaning Policy

The cleaning policy is conservative. Rows are removed only when structural fields are invalid after casting: missing or invalid flight date, month, airline code, origin airport, destination airport, cancelled flag, or diverted flag. Cancelled flights, diverted flights, null delay values, and negative delay values are preserved.

Empty strings are normalized to null. Numeric delay fields are cast to numeric types but are not imputed. This matters because Spark and Hive averages ignore null values, while cancellation-rate calculations still need cancelled flights in the denominator.

The raw data inspection found 92,970 null departure-delay rows and 113,814 null arrival-delay rows. These rows are retained unless they fail a structural rule. Negative delay values are preserved because they represent early departures or arrivals, not data errors.

3.3 Generated Input Sizes

The input-size generator creates controlled benchmark inputs under `data/generated/`:

- 100k
- 500k
- 1m
- 3m
- full
- 14m

Smaller inputs are selected with a deterministic hash-based method using seed 20240520. This avoids chronological bias from simply taking the first rows of the CSV. The 14m input was generated and validated with controlled replication: one complete repetition of the canonical prepared dataset plus a deterministic sampled remainder of 6,920,919 rows. The output keeps the same prepared schema, and no synthetic columns or schema changes are retained.

This replicated larger input is valid scalability evidence because it increases data volume while preserving the canonical row shape, column types, null behavior, and analysis semantics used by the original prepared dataset. It is not treated as new statistical evidence about flight behavior; it is used only to test how the implementations respond to a larger input volume. The optional 28m input remains stretch evidence and was deferred for this milestone.

4 Analyses And Implementations

4.1 Analysis 1: Delay By Airport, Month, And Delay Range

This job satisfies the assignment requirement to report delay behavior for each departure airport and month. It groups rows by:

- `origin_airport`
- `month`
- `delay_range`

The delay ranges are:

- `low: departure_delay < 15`
- `medium: 15 <= departure_delay <= 60`
- `high: departure_delay > 60`

Rows with null departure delay are excluded from this job because they cannot be assigned to one of the required ranges. For each group, the output reports flight count, average departure delay, average arrival delay, and the three most frequent delay or cancellation causes.

The stable output schema includes:

Field group	Columns
Grouping keys	<code>origin_airport</code> , <code>month</code> , <code>delay_range</code>
Metrics	<code>flight_count</code> , <code>avg_departure_delay</code> , <code>avg_arrival_delay</code>
Top causes	<code>top_1_cause</code> , <code>top_1_count</code> , <code>top_2_cause</code> , <code>top_2_count</code> , <code>top_3_cause</code> , <code>top_3_count</code>

Cause labels are derived from cancellation codes or the largest positive delay-cause field. Cause ties are deterministic: cause count descending, then cause label ascending. Groups with fewer than three available causes use an empty cause value and count 0.

Textual pseudocode:

```
filter rows where departure_delay is not null
derive delay_range from departure_delay
derive cause from cancellation code or largest positive delay-cause field
group by origin_airport, month, delay_range
compute flight count and average delays
count causes inside each group
rank causes by count descending and cause label ascending
emit the first three causes and pad missing slots with count 0
order output deterministically
```

4.2 Analysis 2: Airline-Airport Ranking

This job satisfies the assignment requirement to compare each airline at each departure airport against the airport average. For each (`origin_airport`, `airline_code`) pair it computes:

- flight count;
- average departure delay;
- average arrival delay;
- cancellation rate;
- airport average departure delay;
- difference from the airport average;
- rank at the airport, from lowest average departure delay to highest.

Textual pseudocode:

```

group flights by origin_airport and airline_code
compute airline-level counts, averages, and cancellation rate
group flights by origin_airport
compute airport-level average departure delay
join airline metrics with airport metrics
rank airlines per airport by avg_departure_delay ascending, nulls last
order output by airport, rank, average delay, and airline code

```

4.3 Spark SQL

Spark SQL is the reference implementation. It expresses both analyses with DataFrame temporary views, SQL aggregations, and window functions. The ranking job uses `RANK() OVER (PARTITION BY origin_airport ORDER BY avg_departure_delay ASC NULLS LAST)`. The delay job uses grouped cause counts and `ROW_NUMBER()` to select the top three causes with deterministic tie-breaking.

Spark SQL is the most concise implementation in this project. The grouping keys, derived fields, ranking rule, and top-three cause rule are visible in the query structure. The cost is that grouped aggregations and window functions still require shuffles; declarative syntax makes the logic easier to maintain, but it does not remove distributed execution cost.

Execution plan: `make run-spark-sql` reads the canonical prepared Parquet input with the Spark settings from `config/local.yaml`, registers a temporary view, executes one SQL query per analysis, writes full CSV outputs plus first-10 samples under `outputs/spark_sql/`, and records runtime metrics in `outputs/spark_sql/runtime_metrics.json`. Its outputs are validated directly and then used as the correctness reference for Spark Core, Hive, and the MapReduce stretch.

4.4 Spark Core

Spark Core reimplements the Spark SQL logic with RDD transformations. It uses `reduceByKey` for delay aggregates, cause counts, airline aggregates, and airport aggregates. This avoids `groupByKey` for large aggregations and allows partial accumulator state to be combined before the shuffle.

The Spark Core implementation is intentionally explicit. It builds accumulator objects for counts, sums, cancellation totals, and cause counts; joins compact aggregate RDDs by airport; and sorts per-airport airline rows to reproduce the Spark SQL rank semantics. This gives more control over the physical steps but also makes the implementation more verbose and easier to get wrong.

Execution plan: `make run-spark-core` reads the same prepared Parquet input, converts rows into RDD records, runs the two analysis pipelines with keyed aggregations, joins, and deterministic sorts, then materializes the small result tables to the same CSV contract used by Spark SQL. Runtime metrics are written under `outputs/spark_core/`, and validation compares Spark Core row counts, keys, numeric values, top-three causes, and ranking order against Spark SQL.

4.5 Hive

Hive is the third required technology. The runner starts the Docker Compose Hive stack, creates an external table over the prepared Parquet data, executes HiveQL versions of both analyses, and exports CSV outputs with the same schemas as Spark SQL and Spark Core.

Hive is useful as a SQL-on-Hadoop comparison point. Its SQL syntax is familiar for aggregation-heavy work, but this project runs Hive as a local containerized HiveServer2/metastore/PostgreSQL stack. The setup does not use a Hive-on-YARN or HDFS service, so the Hive results should be read as controlled local/container evidence rather than as managed-service Hive performance.

Execution plan: `make run-hive` starts the local Docker Compose Hive services, creates an external Hive table over the prepared Parquet directory, runs the two HiveQL scripts, exports query results through temporary text tables, and normalizes them into the shared CSV output layout under `outputs/hive/`. Runtime metrics

record per-query duration and output rows. Validation compares the exported Hive outputs against the Spark SQL reference.

4.6 MapReduce Stretch

M6 adds Hadoop Streaming MapReduce as an optional stretch, not as part of the three required technology baseline. The implementation uses Python mappers and reducers under `src/mapreduce/`, runs through a Docker-based `mapreduce-runner` service, and writes the same output schemas under `outputs/mapreduce/`.

MapReduce reads a canonical CSV export generated from prepared Parquet. The export is stored under `data/generated/mapreduce_csv/` with a manifest that records the source Parquet signature, row count, CSV size, CSV SHA-256, and export duration. This keeps the stretch comparison fair without reparsing the raw Kaggle CSV, while avoiding stale cached exports.

The delay reducer computes the same grouped counts, averages, and top-three cause ordering as Spark SQL. The ranking reducer groups by airport, aggregates airline statistics, computes the airport average departure delay, and applies Spark SQL `RANK()` semantics with null averages last. Validation passed against Spark SQL for the full prepared dataset; the separate benchmark smoke run uses the 100k generated input.

The supported runtime is Docker-local Hadoop Streaming, not YARN/HDFS. Benchmark smoke runs use isolated output roots under `outputs/mapreduce/.benchmark_runs/`, leaving `outputs/mapreduce/` reserved for validated report-ready outputs.

Execution plan: `make run-mapreduce` exports the selected prepared Parquet input to a canonical CSV with a manifest, runs Hadoop Streaming mapper/reducer jobs inside the Docker-local runtime, sorts and normalizes reducer output into the shared CSV schemas, and records runtime metrics under `outputs/mapreduce/`. The validator compares the stretch outputs against Spark SQL before the report uses them as evidence.

5 Docker Standalone Simulation

The Docker standalone simulation is the repository's controlled alternative execution setting. Spark uses Docker Compose with one Spark standalone master, two Spark workers, and a driver service. The workers are separate containers and processes, but they all run on one physical laptop and share Docker Desktop CPU, memory, and disk limits. Data is read from a bind-mounted project directory, not HDFS or object storage. This is not a true distributed cluster; it is a repeatable single-host simulation of Spark standalone execution.

Hive is included in the Docker benchmark CSV so the report contains rows for all three required technologies, but Hive remains a single-node containerized Hive setup. Therefore the report uses the phrase "Docker standalone simulation" and avoids presenting these runs as future remote-service performance.

The documented topology is stored in `docs/docker_simulation.md`. The command used to run the simulation evidence is:

```
make benchmark-docker-simulation
```

6 Environment And Runtime Configuration

The environment summary was generated by `scripts/generate_environment_summary.py`. The complete artifact is stored at `report/tables/environment_summary.md`.

Category	Item	Value
Host OS	Windows detail	Microsoft Windows 11 Home 10.0.26200 build 26200 64-bit
CPU	Model	AMD Ryzen AI 7 350 w/ Radeon 860M
CPU	Cores	8 physical, 16 logical
Memory	Host RAM	33,598,853,120 bytes

Category	Item	Value
Disk	Project drive	NVMe SSD, 1,024,209,543,168 bytes
Python	Version	3.12.2
Java	Version	OpenJDK 17.0.19
PySpark	Version	4.1.1
Hive	Base image	apache/hive:4.0.1
MapReduce	Base image	apache/hive:4.0.1 with Hadoop Streaming and Python 3
Docker	Version	29.4.3
Docker Compose	Version	v5.1.3
Docker Desktop	Limits	16 CPUs, 16,391,536,640 memory bytes
Local Spark	Master and partitions	local[*], 8 shuffle partitions
Docker Spark	Master and partitions	spark://spark-master:7077, 16 shuffle partitions
Docker topology	Spark	1 master, 2 workers, 2 cores and 2 GB memory per worker
Docker topology	Hive	HiveServer2, metastore, PostgreSQL, single host

7 Benchmark Evidence

The benchmark runner records technology, job name, input size, environment, execution-setting label, repetition number, raw duration, output rows, status, timestamp, input path, and metrics path. By default, each selected input/technology configuration is run three times. Timestamped benchmark CSVs remain raw audit evidence, while the report-ready artifacts under `report/tables/` and `report/figures/` aggregate successful repetitions.

The M2 reporting pipeline computes median, mean, minimum, maximum, and standard deviation of duration for each environment/input/job/technology group. Existing single-run CSVs are still readable as `runs=1`, and fresh benchmark campaigns can be forced to one run for smoke checks with `BENCHMARK_FLAGS="--repetitions 1"`. M6 adds an opt-in MapReduce benchmark smoke so the stretch can be included without slowing the required default benchmark matrix:

Environment	Inputs	Technologies	Jobs	Status
Local	100k, 500k, 1m, 3m, full	Spark SQL, Spark Core, Hive	both jobs	30/30 successful
Local M3 large input	14m	Spark SQL, Spark Core	both jobs	12/12 successful
Docker standalone simulation	100k, 500k, 1m	Spark SQL, Spark Core, Hive	both jobs	18/18 successful
Local stretch	100k	Hadoop Streaming MapReduce	both jobs	2/2 successful

The local run ID is 20260521T153035251181Z. The Docker standalone simulation run ID is 20260521T160042313560Z. The MapReduce smoke run ID is 20260521T153912538439Z. The M3 14m run ID is 20260521T220744784794Z. These legacy campaigns are represented as one-run aggregate rows until the benchmark matrix is regenerated with the M2 default repetition count; the 14m Spark SQL and Spark Core rows use three repetitions. The full status matrix is stored in `report/tables/benchmark_status.md`.

7.1 Benchmark Pivot

For compactness, the printed pivot below focuses on median duration for the three required technologies. The regenerated `report/tables/benchmark_pivot.*` artifacts also include the optional MapReduce smoke columns when present.

environment	input_label	records	job_name	Spark SQL median s	Spark Core median s	MapReduce smoke s
docker-simulation	100k	100000	airline_airport_ranking	4.01	2.325	8.125
docker-simulation	100k	100000	delay_by_airport_month	9.385	2.543	1.015

environment	input_label	records	job_name	Spark SQL median s	Spark Core median s	F
docker-simulation	500k	500000	airline_airport_ranking	5.617	2.434	8
docker-simulation	500k	500000	delay_by_airport_month	9.978	2.623	1
docker-simulation	1m	1000000	airline_airport_ranking	4.444	2.334	8
docker-simulation	1m	1000000	delay_by_airport_month	10.921	2.619	1
local	100k	100000	airline_airport_ranking	1.056	0.57	8
local	100k	100000	delay_by_airport_month	5.215	0.916	1
local	500k	500000	airline_airport_ranking	1.104	0.592	8
local	500k	500000	delay_by_airport_month	5.74	0.873	1
local	1m	1000000	airline_airport_ranking	1.31	0.563	8
local	1m	1000000	delay_by_airport_month	5.901	0.812	1
local	3m	3000000	airline_airport_ranking	1.659	0.615	8
local	3m	3000000	delay_by_airport_month	6.662	0.935	1
local	full	7079081	airline_airport_ranking	2.33	0.583	1
local	full	7079081	delay_by_airport_month	7.641	0.962	1
local	14m	14000000	airline_airport_ranking	2.241	0.564	
local	14m	14000000	delay_by_airport_month	7.68	0.859	

7.2 Rows Per Second

Rows-per-second values are computed as input records divided by median elapsed seconds. They show that fixed startup and planning costs dominate small inputs: throughput often increases sharply even when median elapsed seconds change only slightly.

environment	input	job	Spark SQL rows/s	Spark Core rows/s	Hive rows/s
docker-simulation	100k	airline_airport_ranking	24940.511	43008.699	11131.84
docker-simulation	100k	delay_by_airport_month	10654.924	39330.733	8315.777
docker-simulation	500k	airline_airport_ranking	89016.757	205410.597	55729.003
docker-simulation	500k	delay_by_airport_month	50111.307	190653.625	39622.019
docker-simulation	1m	airline_airport_ranking	225042.606	428401.292	112352.734
docker-simulation	1m	delay_by_airport_month	91570.396	381793.636	79463.103
local	100k	airline_airport_ranking	94656.991	175531.597	11206.355
local	100k	delay_by_airport_month	19175.216	109155.648	8380.722
local	500k	airline_airport_ranking	452990.462	844253.756	56732.374
local	500k	delay_by_airport_month	87100.791	572454.41	39393.811
local	1m	airline_airport_ranking	763139.74	1775987.582	112859.06
local	1m	delay_by_airport_month	169476.703	1231376.963	78843.778
local	3m	airline_airport_ranking	1807870.383	4878413.67	338549.593
local	3m	delay_by_airport_month	450294.74	3207389.826	217435.311
local	full	airline_airport_ranking	3038232.189	12144151.609	571725.628
local	full	delay_by_airport_month	926412.613	7357534.241	368709.471
local	14m	airline_airport_ranking	6247855.089	24825820.272	
local	14m	delay_by_airport_month	1823023.01	16295744.483	

7.3 Speedup Ratios

Each value is a median-duration ratio. A value above 1 means the numerator took longer than the denominator for the aggregate row.

environment	input	job	SQL/Core	Hive/SQL	Hive/Core
docker-simulation	100k	airline_airport_ranking	1.724	2.24	3.864
docker-simulation	100k	delay_by_airport_month	3.691	1.281	4.73
docker-simulation	500k	airline_airport_ranking	2.308	1.597	3.686
docker-simulation	500k	delay_by_airport_month	3.805	1.265	4.812
docker-simulation	1m	airline_airport_ranking	1.904	2.003	3.813
docker-simulation	1m	delay_by_airport_month	4.169	1.152	4.805
local	100k	airline_airport_ranking	1.854	8.447	15.664
local	100k	delay_by_airport_month	5.693	2.288	13.025
local	500k	airline_airport_ranking	1.864	7.985	14.881
local	500k	delay_by_airport_month	6.572	2.211	14.532
local	1m	airline_airport_ranking	2.327	6.762	15.736
local	1m	delay_by_airport_month	7.266	2.15	15.618
local	3m	airline_airport_ranking	2.698	5.34	14.41
local	3m	delay_by_airport_month	7.123	2.071	14.751
local	full	airline_airport_ranking	3.997	5.314	21.241
local	full	delay_by_airport_month	7.942	2.513	19.955

7.4 Normalized Scalability

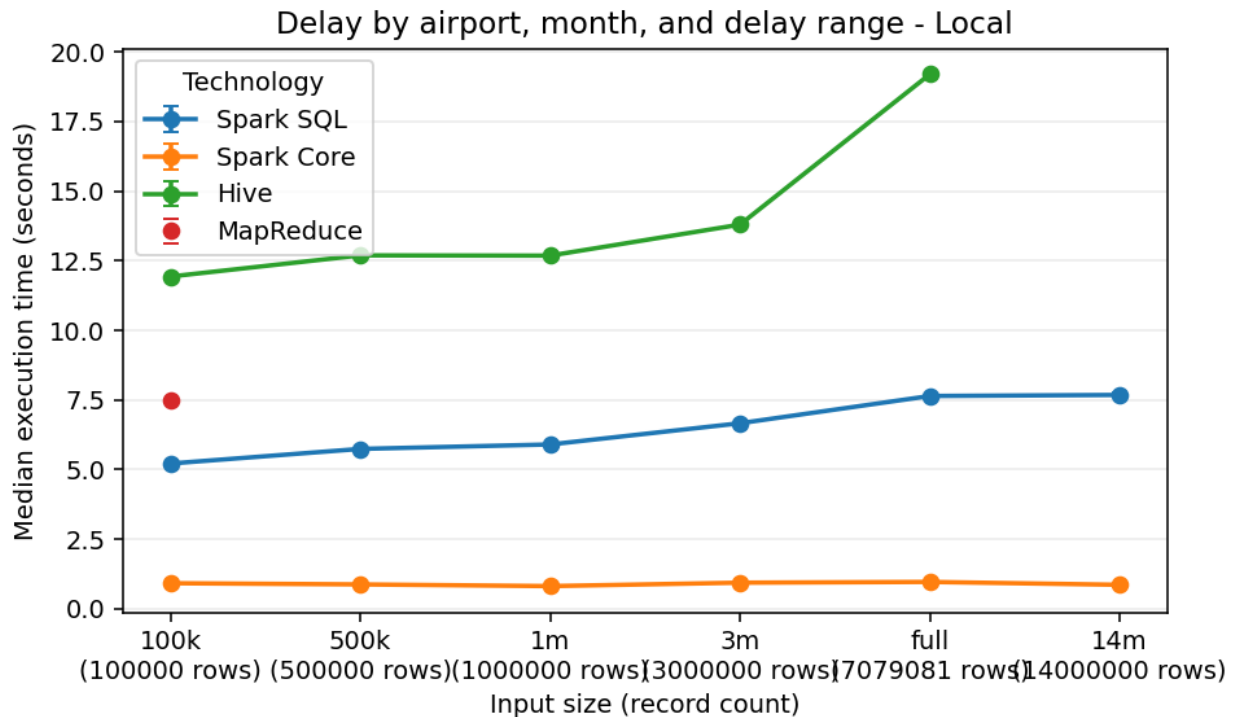
The following table normalizes median duration, record count, and throughput against the 100k baseline for each environment, job, and technology. For example, a `records_vs_100k` value of 10 means the input has ten times as many records as the baseline. The compact columns `dur`, `rec`, and `thr` mean `median_duration_vs_100k`, `records_vs_100k`, and `throughput_vs_100k`.

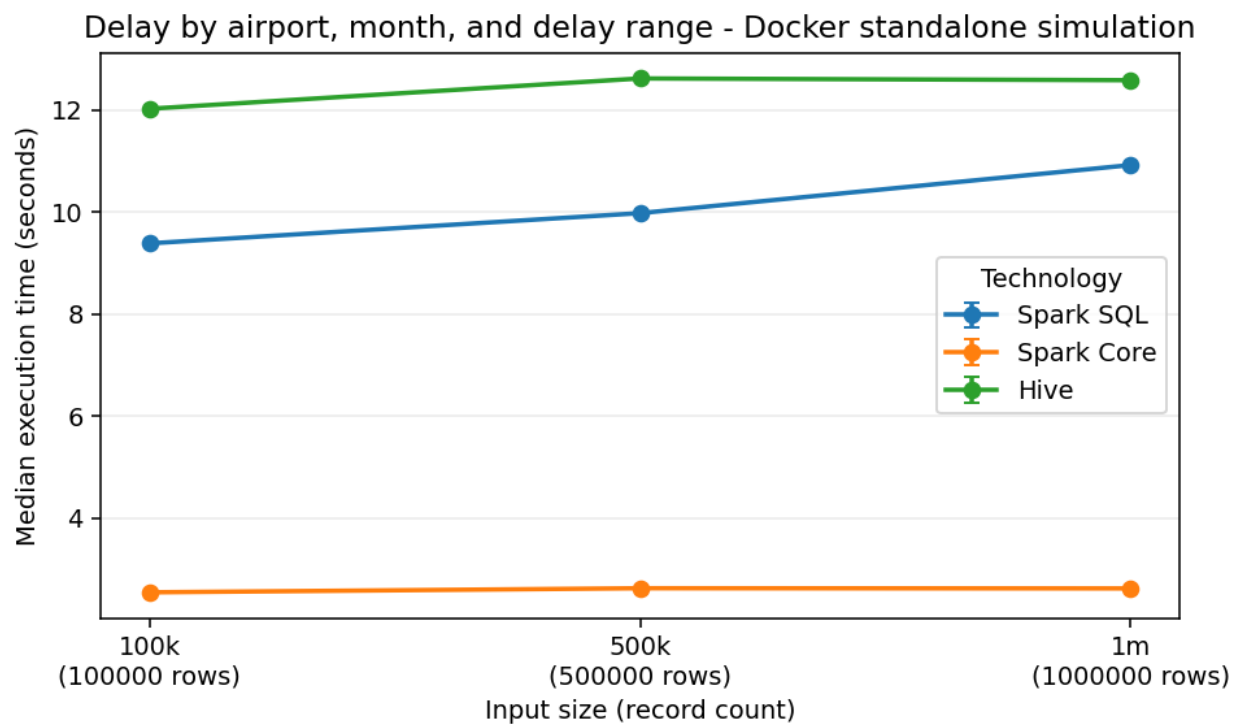
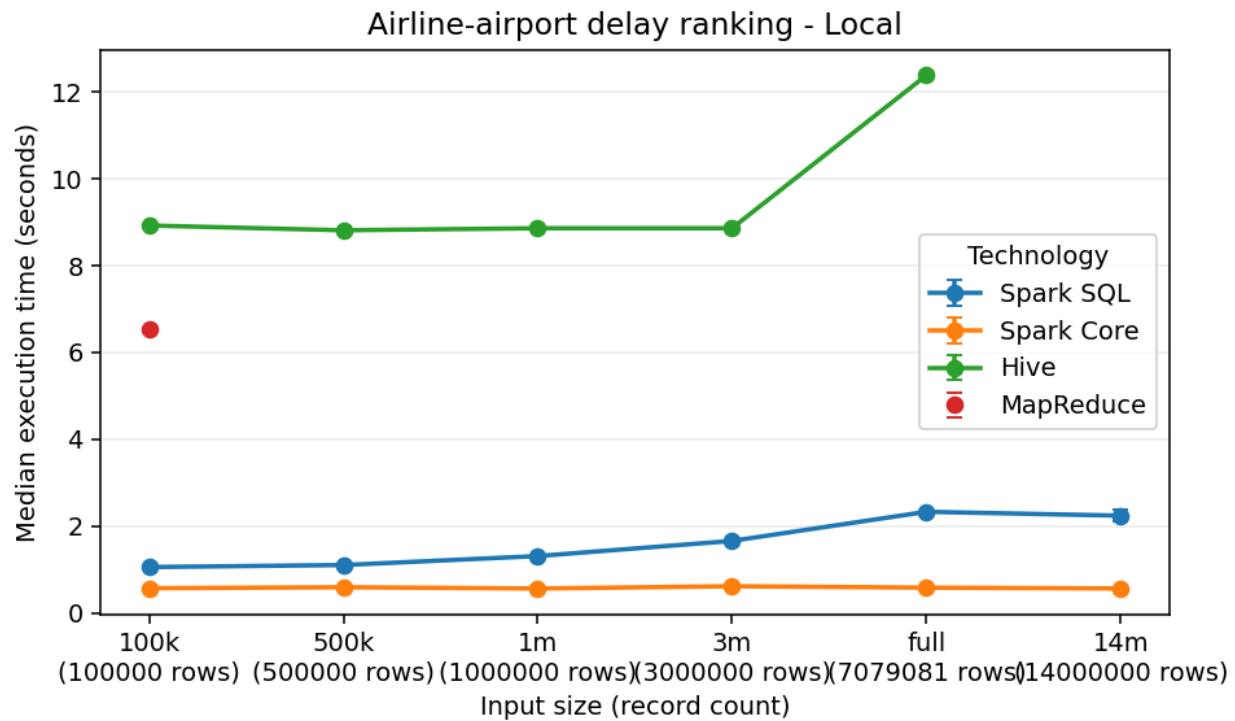
env	input	job	tech	dur	rec	thr
docker	100k	ranking	Hive	1	1	1
docker	500k	ranking	Hive	0.999	5	5.006
docker	1m	ranking	Hive	0.991	10	10.093
docker	100k	ranking	Core	1	1	1
docker	500k	ranking	Core	1.047	5	4.776
docker	1m	ranking	Core	1.004	10	9.961
docker	100k	ranking	SQL	1	1	1
docker	500k	ranking	SQL	1.401	5	3.569
docker	1m	ranking	SQL	1.108	10	9.023
docker	100k	delay	Hive	1	1	1
docker	500k	delay	Hive	1.049	5	4.765
docker	1m	delay	Hive	1.046	10	9.556
docker	100k	delay	Core	1	1	1
docker	500k	delay	Core	1.031	5	4.847
docker	1m	delay	Core	1.03	10	9.707
docker	100k	delay	SQL	1	1	1
docker	500k	delay	SQL	1.063	5	4.703
docker	1m	delay	SQL	1.164	10	8.594
local	100k	ranking	Hive	1	1	1
local	500k	ranking	Hive	0.988	5	5.063
local	1m	ranking	Hive	0.993	10	10.071
local	3m	ranking	Hive	0.993	30	30.21
local	full	ranking	Hive	1.388	70.791	51.018
local	100k	ranking	Core	1	1	1
local	500k	ranking	Core	1.04	5	4.81
local	1m	ranking	Core	0.988	10	10.118

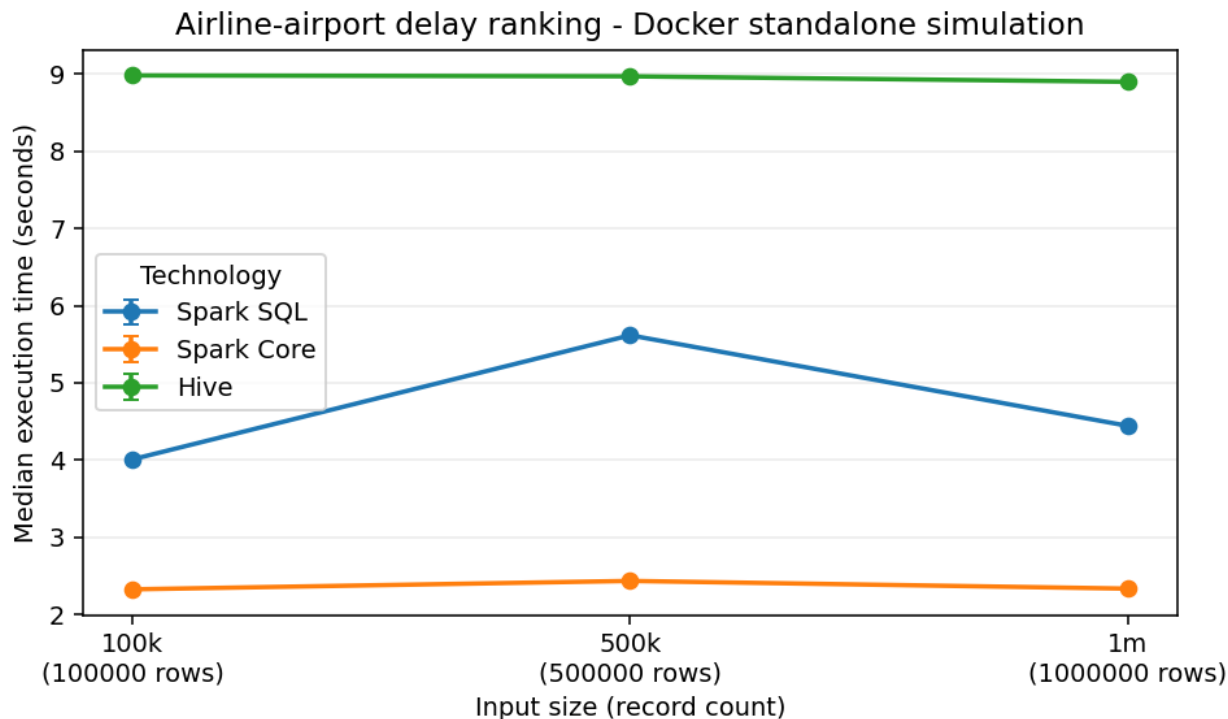
env	input	job	tech	dur	rec	thr
local	3m	ranking	Core	1.079	30	27.792
local	full	ranking	Core	1.023	70.791	69.185
local	100k	ranking	SQL	1	1	1
local	500k	ranking	SQL	1.045	5	4.786
local	1m	ranking	SQL	1.24	10	8.062
local	3m	ranking	SQL	1.571	30	19.099
local	full	ranking	SQL	2.206	70.791	32.097
local	100k	delay	Hive	1	1	1
local	500k	delay	Hive	1.064	5	4.701
local	1m	delay	Hive	1.063	10	9.408
local	3m	delay	Hive	1.156	30	25.945
local	full	delay	Hive	1.609	70.791	43.995
local	100k	delay	Core	1	1	1
local	500k	delay	Core	0.953	5	5.244
local	1m	delay	Core	0.886	10	11.281
local	3m	delay	Core	1.021	30	29.384
local	full	delay	Core	1.05	70.791	67.404
local	100k	delay	SQL	1	1	1
local	500k	delay	SQL	1.101	5	4.542
local	1m	delay	SQL	1.131	10	8.838
local	3m	delay	SQL	1.278	30	23.483
local	full	delay	SQL	1.465	70.791	48.313

7.5 Benchmark Charts

Execution-time charts are generated separately for local execution and Docker standalone simulation. Charts plot median duration and include min/max variability indicators when repeated successful measurements are available. Line plots are used only when at least three input sizes are available for a job and environment.







8 AWS EMR Cluster Experiment

The project now includes real Amazon EMR evidence in addition to local and Docker standalone simulation runs. The baseline EMR profile is the completed run `m4-emr-final-2`: EMR `emr-7.13.0` in `us-east-1`, Spark on YARN, one primary node and two core nodes, all using `m5.xlarge` instances. That run covered 100k, 500k, 1m, 3m, full, and 14m inputs for Spark SQL and Spark Core, with S3 used for uploaded inputs, source bundles, logs, metrics, and downloaded outputs.

8.1 Cluster Size Variation

Milestone 5 uses `m4-emr-final-2` as the baseline cluster-size point and `m5-emr-3core-1m-full` as the larger-cluster point. The larger run used one primary node and three core nodes, the same `m5.xlarge` instance type, the same EMR release, and the same Spark/YARN execution path. It completed on cluster `j-LTX1FIHYB4X9` from 2026-05-22T00:46:03Z to 2026-05-22T01:12:14Z, with an estimated cost of 0.3944 USD. The larger-profile matrix was intentionally limited to 1m and full so the evidence stayed within AWS Academy Learner Lab budget limits. Docker standalone simulation is shown only where matching data exists; the Docker benchmark matrix currently stops at 1m, so Docker full cells are marked N/A instead of being overclaimed.

The generated cluster-size comparison table is stored in `report/tables/cluster_size_comparison.md`. Compared with the baseline EMR profile, the larger profile improved Spark Core medians in all four comparable cells, with speedups from 1.146x to 1.221x. Spark SQL was mixed: it improved slightly for the 1m delay job (1.052x) but was slightly slower for 1m ranking, full delay, and full ranking (0.951x to 0.991x). This is still useful scalability evidence because it shows that adding a core node does not uniformly accelerate these short analytical jobs; fixed Spark startup, S3 file listing and reads, shuffle planning, and small output cardinality remain large parts of the measured runtime. These results should be interpreted as limited scalability evidence because clusters are short-lived, S3 I/O is part of the managed execution path, repetitions are constrained by budget, and Learner Lab service limits prevent wider cluster-size sweeps.

9 Critical Discussion

Spark SQL is the most expressive implementation. It is especially strong for the two selected analyses because both are naturally relational: they derive columns, group by keys, aggregate measures, and rank rows inside partitions. Window functions make the airline ranking and top-three cause selection clear. The tradeoff is that the physical plan still has to shuffle data for grouped aggregations and window partitions. Spark SQL hides some mechanics, but it does not make those costs disappear.

Spark Core is the most explicit implementation. The RDD version exposes every map, key construction, accumulator merge, join, and sort. Its accumulator design is useful because it combines counts and sums before the shuffle and keeps the top-three cause logic deterministic. That explicitness also makes the code more verbose. Reproducing SQL ranking semantics, null handling, and top-three padding requires careful manual logic that Spark SQL expresses more directly.

Hive provides SQL-on-Hadoop style expressiveness, but it has the highest overhead in this project. The containerized Hive stack pays service, query startup, metastore, and export costs that are visible in the benchmark rows. Hive remains useful as a third technology and as a contrast with Spark, but the current local container setup does not demonstrate the strengths of a managed Hive service.

MapReduce is the most manual implementation. Hadoop Streaming makes the map and reduce boundaries explicit, and the Python reducers show the accumulator state required for averages, ranks, and top-three causes. The tradeoff is extra orchestration: prepared Parquet must be exported to canonical CSV, jobs must be run through Hadoop Streaming, and reducer output must be normalized back into the common CSV contract. This makes MapReduce valuable as a stretch comparison, but less ergonomic than Spark SQL for these analyses.

The delay-by-airport-month job is generally more expensive than the airline-airport ranking. It derives delay ranges, maps cancellation or delay causes, counts causes inside each airport-month-range group, and ranks causes to emit three positions. These steps increase shuffle and aggregation pressure. The ranking job also performs grouped aggregation and per-airport ranking, but after the first aggregation it works with compact airport-airline summaries.

Data preparation affects both correctness and benchmark fairness. Correctness improves because each technology reads the same typed Parquet schema instead of reimplementing raw CSV parsing. Benchmark fairness improves because all technologies operate on the same generated inputs. The limitation is that preparation cost is not included in the per-job execution-time tables; it is a separate mandatory pipeline stage documented in this report.

Small input sizes can distort runtime comparisons. JVM startup, Spark session startup, Docker container/service setup, Hive query startup, file listing, and query planning are fixed costs. Reads from prepared Parquet can also benefit from compact columnar layout and operating-system cache effects, while both analyses emit relatively small aggregate outputs compared with the input size. At 100k, these effects can dominate elapsed time; at larger sizes, throughput may improve even when raw seconds are flat or non-monotonic. The M2 workflow therefore reports median duration and aggregate timing statistics when repeated runs are available, while preserving raw per-repetition CSV rows for audit.

10 Reproducibility And Validation

Generated large data, raw data, and full output directories are ignored by Git. The repository commits only small evaluator-facing evidence: report tables, figures, source code, documentation, and scripts.

Correctness validation is available through:

```
.\.venv\Scripts\python.exe scripts\validate_spark_sql_outputs.py
.\.venv\Scripts\python.exe scripts\validate_spark_core_outputs.py
.\.venv\Scripts\python.exe scripts\validate_hive_outputs.py
.\.venv\Scripts\python.exe scripts\validate_mapreduce_outputs.py
```

Spark Core, Hive, and MapReduce validators compare their outputs against Spark SQL, including output columns, row counts, key sets, numeric values within tolerance, top-three cause labels and counts, ranking order, and first-10 sample files.

11 Limitations

- The raw dataset and generated benchmark inputs are not committed to Git because they are large.
- The raw dataset does not include airline names, so analyses use `airline_code`.
- MapReduce depends on Docker-based Hadoop Streaming and is kept outside the default `make run-all-local` and `make benchmark-local` paths.
- The Docker Spark setup is a Docker standalone simulation on one physical machine, not a true distributed cluster.
- Hive is containerized locally and is not running on a Hadoop/YARN service.
- Worker-count variation was not used in M2 because the reliable Docker Compose topology has two named Spark workers.
- Windows Spark runs require care around Java, Hadoop `winutils.exe`, and native file handling; the project includes compatibility helpers and Docker paths where needed.
- Benchmark results are hardware-dependent and should be interpreted as evidence for this controlled environment, not universal performance numbers.

12 Conclusion

The project satisfies the main assignment requirements with a reproducible data preparation pipeline, two completed analyses, three technology implementations, first-10 result samples for each job and technology, benchmark tables, execution-time charts, and a critical discussion of expressiveness, implementation effort, efficiency, scalability, shuffle, aggregation, and data preparation effects. Spark SQL provides the clearest reference logic, Spark Core demonstrates lower-level distributed transformation control, and Hive provides the required SQL-on-Hadoop comparison point. The completed MapReduce stretch adds a fourth validated implementation and shows the extra mechanics needed when using Hadoop Streaming directly.

13 Evidence Appendix: First 10 Result Rows

The tables below embed the first 10 rows generated for both analytical jobs and the three required technologies. Wide outputs are split into paired narrow tables so the PDF remains readable. The source artifacts are stored under `report/tables/` and are regenerated by `make charts`. MapReduce stretch sample artifacts are generated with the same naming convention and summarized after the required-technology samples.

13.1 Spark SQL Samples

13.1.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
--------	-----	-------	---------	----------	----------

13.1.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

13.1.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

13.1.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

13.2 Spark Core Samples

13.2.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

13.2.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

13.2.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

13.2.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

13.3 Hive Samples

13.3.1 Delay Metrics

Origin	Mo.	Range	Flights	Avg dep.	Avg arr.
ABE	1	high	8	231.5	221.375
ABE	1	low	35	-6.286	-19.629
ABE	1	medium	4	32.75	31.25
ABE	2	high	3	324	335.667
ABE	2	low	40	-8.5	-23.575
ABE	2	medium	4	45.5	29.333
ABE	3	high	8	127	121.625
ABE	3	low	41	-4.756	-16.39
ABE	3	medium	3	32.333	12.667
ABE	4	high	6	158.333	150.333

13.3.2 Delay Top-Three Causes

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	1	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	1	low	unknown (34)	delay:nas (1)	none (0)
ABE	1	medium	delay:late_aircraft (2)	delay:carrier (1)	delay:nas (1)
ABE	2	high	delay:late_aircraft (1)	delay:nas (1)	delay:weather (1)
ABE	2	low	unknown (39)	delay:nas (1)	none (0)

Origin	Mo.	Range	Top 1	Top 2	Top 3
ABE	2	medium	unknown (2)	delay:carrier (1)	delay:late_aircraft (1)
ABE	3	high	delay:carrier (3)	delay:late_aircraft (3)	delay:nas (2)
ABE	3	low	unknown (40)	delay:nas (1)	none (0)
ABE	3	medium	unknown (2)	delay:carrier (1)	none (0)
ABE	4	high	delay:carrier (4)	delay:late_aircraft (2)	none (0)

13.3.3 Ranking Metrics

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	9E	140	10.234	0.445	0.021
ABE	G4	260	12.463	4.337	0.012

Origin	Airline	Flights	Avg dep.	Avg arr.	Cancel rate
ABE	OH	159	19.38	6.867	0.006
ABE	OO	40	25.297	14.389	0.075
ABI	MQ	249	6.664	4.405	0.008
ABQ	DL	218	1.383	-5.15	0.018
ABQ	OO	547	3.617	-0.855	0.004
ABQ	UA	273	4.584	-1.477	0.022
ABQ	MQ	127	7.134	5.575	0
ABQ	AS	67	7.522	1.881	0

13.3.4 Ranking Comparison

Origin	Airline	Airport avg dep.	Diff vs airport	Rank
ABE	9E	14.606	-4.373	1
ABE	G4	14.606	-2.143	2
ABE	OH	14.606	4.774	3
ABE	OO	14.606	10.691	4
ABI	MQ	6.664	0	1
ABQ	DL	8.792	-7.409	1
ABQ	OO	8.792	-5.176	2
ABQ	UA	8.792	-4.208	3
ABQ	MQ	8.792	-1.659	4
ABQ	AS	8.792	-1.27	5

13.4 MapReduce Stretch Samples

The M6 stretch implements both selected analyses with Hadoop Streaming MapReduce. It is optional evidence after the required Spark SQL, Spark Core, and Hive samples above. The validated full prepared-data run produced 11,902 delay rows and 1,738 ranking rows, matching the Spark SQL reference on keys, numeric fields, rank values, and top-three cause labels/counts.

Artifact	Path
Runtime metrics	outputs/mapreduce/runtime_metrics.json
Delay sample	report/tables/first_10_mapreduce_delay_by_airport_month.md
Ranking sample	report/tables/first_10_mapreduce_airline_airport_ranking.md
Validator	scripts/validate_mapreduce_outputs.py
Runtime documentation	docs/mapreduce_analyses.md

The MapReduce benchmark smoke run on 100k input completed both jobs successfully using isolated benchmark outputs. The benchmark rows are included in generated report tables when present, but missing MapReduce cells are not added to the required Spark/Hive/Spark Core benchmark status matrix.